



Programmation orientée objet : Java

POO: PROGRAMMATION ORIENTÉE OBJET

SURCHARGE DE MÉTHODES

- Dans une même classe, plusieurs méthodes peuvent posséder le même nom, pourvu qu'elles diffèrent en nombre et/ou type de paramètres.
- Une méthode est déterminée par ce que l'on appelle sa **signature**:
 - son nom
 - ses paramètres
- On parle de **surcharge**, on encore en anglais d'**overloading** en anglais.
- Le choix de la méthode à utiliser est fonction des paramètres passés à l'appel.
- Très souvent les constructeurs sont surchargés

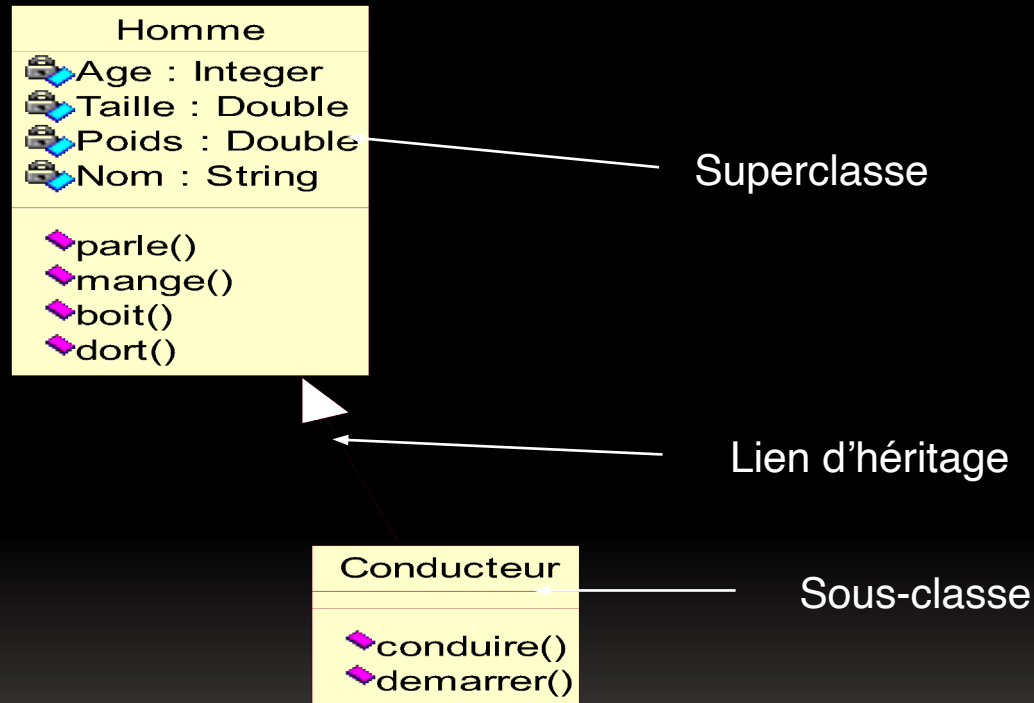
SURCHARGE DE MÉTHODES : EXEMPLE

```
int somme( int p1, int p2){  
    return (p1 + p2);  
}  
float somme( float p1, float p2){  
    return (p1 + p2);  
}  
float somme( float p1, float p2, float p3){  
    return (p1 + p2 + p3);  
}  
int somme( float p1, int p2){  
    return ((int)p1 + p2);  
}
```

L'HÉRITAGE : DÉFINITION

- **Héritage** : mécanisme permettant le partage et la réutilisation de propriétés entre les objets. La relation d'héritage est une relation de généralisation / spécialisation.
 - La classe parente est la **superclasse**.
 - La classe qui hérite est la **sous-classe**.

L'HÉRITAGE : REPRÉSENTATION GRAPHIQUE



Représentation avec UML d'un héritage

L'HÉRITAGE EN JAVA

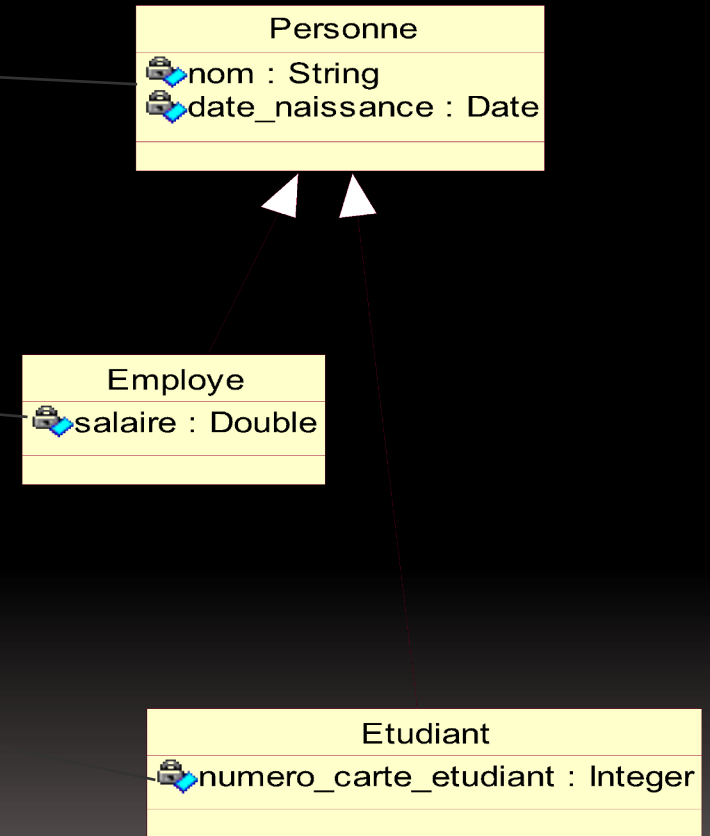
- Java implémente le mécanisme d'héritage simple qui permet est une relation de généralisation / spécialisation entre deux classes.
- Pour le programmeur, il s'agit d'indiquer, dans la sous-classe, le nom de la superclasse dont elle hérite.
- Par défaut toutes classes Java hérite de la classe **Object**.
- L'héritage multiple n'existe pas en Java.
- Mot réservé : **extends**

L'HÉRITAGE EN JAVA

```
class Personne
{
    private String nom;
    private Date date_naissance;
    // ...
}

class Employe extends Personne
{
    private float salaire;
    // ...
}

class Etudiant extends Personne
{
    private int numero_carte_etudiant;
    // ...
}
```



L'HÉRITAGE ET L'ENCAPSULATION

- Le modificateurs d'accès « **protected** » est utilisé pour protéger l'accessibilité des variables et des méthodes en permettant au sous calsse d'accéder.
- Pour résumer les accès sont contrôlés en respectant le tableau suivant:

Mot-clé	classe	package	sous classe	world
private	Y			
protected	Y	Y	Y	
public	Y	Y	Y	Y
[aucun]	Y	Y		

Seul les membres publics sont visibles depuis le monde extérieur.

Une classe a toujours accès à ses membres.

Les classes d'un même package protègent uniquement leurs membres privés (à l'intérieur du package)

Une classe fille (ou dérivée) n'a accès qu'aux membres publics et protected de la classe mère.

L'HÉRITAGE EN JAVA

- Constructeurs et héritage
 - par défaut le constructeur d'une sous-classe appelle le constructeur "par défaut" (celui qui ne reçoit pas de paramètres) de la superclasse. Attention donc dans ce cas que le constructeur sans paramètre existe toujours dans la superclasse...
 - Pour forcer l'appel d'un constructeur précis, on utilisera le mot réservé **super**. Cet appel devra être **la première instruction** du constructeur.

L'HÉRITAGE EN JAVA

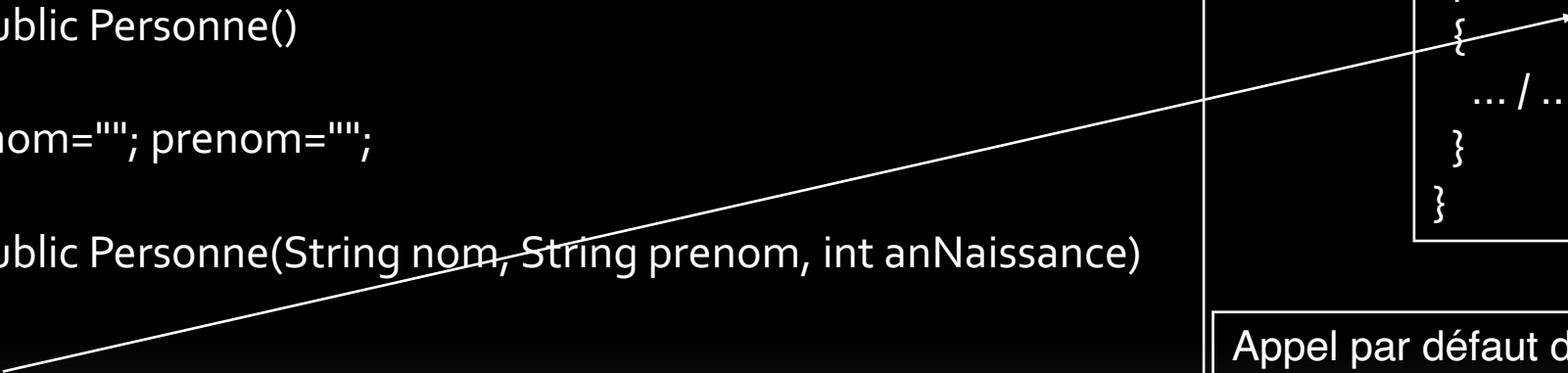
```
public class Employe extends Personne
{
    public Employe () {}
    public Employe (String nom,
                    String prenom,
                    int anNaissance)
    {
        super(nom, prenom, anNaissance);
    }
}
```

Appel explicite à ce constructeur
avec le mot clé super

```
public class Personne
{
    public String nom, prenom;
    public int anNaissance;
    public Personne()
    {
        nom=""; prenom="";
    }
    public Personne(String nom,
                    String prenom,
                    int anNaissance)
    {
        this.nom=nom;
        this.prenom=prenom;
        this.anNaissance=anNaissance;
    }
}
```

L'HÉRITAGE EN JAVA

```
public class Personne
{
    public String nom, prenom;
    public int anNaissance;
    public Personne()
    {
        nom=""; prenom="";
    }
    public Personne(String nom, String prenom, int anNaissance)
    {
        this.nom=nom;
        this.prenom=prenom;
        this.anNaissance=anNaissance;
    }
}
```

A white arrow originates from the opening curly brace of the `Personne` class and points towards the opening curly brace of the `Object` class, indicating that `Personne` inherits from `Object`.

```
public class Object
{
    public Object()
    {
        ... / ...
    }
}
```

Appel par défaut dans le constructeur de `Personne` au constructeur par défaut de la superclasse de `Personne`, qui est `Object`

RÉFÉRENCE À LA SUPERCLASSE

- Le mot réservé **super** permet de faire référence au constructeur de la superclasse directe mais aussi à d'autres informations provenant de cette superclasse.

```
class Employe extends Personne
{
    private float salaire;
    public float calculePrime()
    {
        return (salaire * 0,05);
    }
    // ...
}
```

Appel à la méthode `calculPrime()`
de la superclasse de Cadre

```
class Cadre extends Employe
{
    public float calculePrime()
    {
        return (super.calculPrime() / 2);
    }
    // ...
}
```

REDÉFINITION DE MÉTHODES

- Une sous-classe peut redéfinir des méthodes existant dans une de ses superclasses (directe ou indirectes), à des fins de spécialisation.
 - Le terme anglophone est "overriding". On parle aussi de masquage.
 - La méthode redéfinie **doit avoir la même signature**.

```
class Employe extends Personne
{
    private float salaire;
    public calculePrime()
    {
        // ...
    }
}
```

redéfinition

```
class Cadre extends Employe
{
    public calculePrime()
    {
        // ...
    }
    // ...
}
```



TP